Mini Assessment — API Error Handling

Name: Y Ravi Teja

Reg No: 99230040828

Email: 99230040828@klu.ac.in

Question 1: If an API returns 429, what should your program do?

A 429 status code means "Too Many Requests" — the client has exceeded the API's rate limit. The program should not retry immediately. Instead, it should back off and wait before retrying, ideally reading the **Retry-After** header if the API provides one. Using exponential backoff with a maximum retry limit helps avoid overwhelming the server and prevents the program from getting stuck in an infinite retry loop.

Question 2: If response.json() fails, what should your program do?

A failure here usually means the response body is not valid JSON — for example, the server returned an HTML error page or an empty body instead of JSON. The program should wrap the call in a **try/except** block to catch the exception, then check the response status code and raw text (response.text) for debugging. Instead of crashing, it should handle the error gracefully — for instance, by logging the issue, retrying, or returning a safe default value.

Question 3: If a JSON key is missing, why might .get() be safer than []?

Accessing a dictionary with square brackets, such as data['key'], raises a **KeyError** and crashes the program if the key does not exist. Using data.get('key') instead returns **None** (or a specified default value) when the key is missing, allowing the program to continue running smoothly. For example: data.get('key', 'default_value') safely handles missing data without interrupting program execution.

— End of Assignment —